DD: BIL-02-STA-01 — Status Management & Recovery

Developer implementation guide:
`DD_BIL-02-STA-01-Status_Management_and_Recovery-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
`DD_BIL-02-STA-01-Status_Management_and_Recovery-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.0 Satellite of **BIL-02 Invoicing (Core)**. Owns the post-issuance lifecycle of invoices — the state machine that moves invoices through PENDING / PAID / OVERDUE / DISPUTED / CANCELLED / WRITTEN_OFF, the overdue scanner, promise-to-pay (PTP) handling, dispute handling, and admin-driven recovery operations. Consumes payment events from BIL-01 Charging Engine. Emits lifecycle events consumed by READ-01, BIL-04 Collections, DD_NOT-01 Notification Service, and downstream analytics. **Sibling DDs**: BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API. **Version history**: see `CHANGELOG_BIL-02-STA-01-Status_Management_and_Recovery.md` in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-02-STA-01-Status_Management_and_Recovery-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `GET /api/admin/scanners/runs`
- `GET /api/admin/scanners/{name}/status`
- `GET /api/disputes/{id}`
- `GET /api/disputes?invoice_id={id}`

- `GET /api/ptps/{id}`
- `GET /api/ptps?customer=me`
- `GET /api/ptps?invoice_id={id}`
- `POST /api/admin/disputes/dsp_01HZ.../resolve`
- `POST /api/admin/disputes/{id}/resolve`
- `POST /api/admin/disputes/{id}/start-review`
- `POST /api/admin/invoices/inv_01HZ.../write-off`
- `POST /api/admin/invoices/{id}/force-cancel`
- `POST /api/admin/invoices/{id}/mark-paid-offline`
- `POST /api/admin/invoices/{id}/override-overdue`
- `POST /api/admin/invoices/{id}/restore`
- `POST /api/admin/invoices/{id}/transition`
- `POST /api/admin/invoices/{id}/write-off`
- `POST /api/admin/ptps`
- `POST /api/admin/ptps/{id}/cancel`
- `POST /api/admin/scanners/{name}/trigger`
- `POST /api/disputes`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `billing-invoicing`
- `billing-invoicing-status`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `InvoiceCancelled`
- `InvoiceDisputed`
- `InvoiceIssued`
- `InvoiceManualTransition`
- `InvoiceOverdue`
- `InvoiceOverdueOverridden`

- `InvoicePaid`
- `InvoicePartiallyPaid`
- `InvoiceRestored`
- `InvoiceStatusChanged`
- `InvoiceWrittenOff`
- `PaymentApplied`
- `PaymentReceived`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.0

Satellite of **BIL-02 Invoicing (Core)**. Owns the post-issuance lifecycle of invoices — the state machine that moves invoices through PENDING / PAID / OVERDUE / DISPUTED / CANCELLED / WRITTEN_OFF, the overdue scanner, promise-to-pay (PTP) handling, dispute handling, and admin-driven recovery operations. Consumes payment events from BIL-01 Charging

Engine. Emits lifecycle events consumed by READ-01, BIL-04 Collections, DD_NOT-01 Notification Service, and downstream analytics.

Sibling DDs: BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API.

Version history: see `CHANGELOG_BIL-02-STA-01-Status_Management_and_Recovery.md` in the same directory.

Glossary

Term	Meaning
Lifecycle status	The current state of an invoice in its post-issuance journey: `PENDING`, `PAID`, `PARTIALLY_PAID`, `OVERDUE`, `DISPUTED`, `CANCELLED`, `WRITTEN_OFF`. Stored as `invoice.status` (column owned by BIL-02 Core); transitions managed by STA-01.
State transition	A move from one status to another, triggered by an event (payment received, due date passed) or an admin action (force-cancel). Captured in `invoice_history`. STA-01 enforces which transitions are allowed.
Overdue	An invoice whose `due_date` has passed and which has not been fully paid. STA-01's overdue scanner flips PENDING → OVERDUE (or PARTIALLY_PAID → OVERDUE_PARTIAL).
Promise-to-pay (PTP)	A formal commitment by a customer to pay an outstanding invoice by a specific date (or in installments). First-class entity in `promise_to_pay`. While an active PTP exists, the invoice is shielded from overdue transition and dunning escalation until the PTP date expires.
Dispute	A formal customer complaint that an invoice (or specific lines) is incorrect. First-class entity in `dispute`. While an active dispute exists, the invoice is in DISPUTED status; enforcement is paused; resolution requires admin action (uphold = invoice valid, return to prior status; resolve-in-favor-of-customer = trigger adjustment via ADJ-01).
Recovery operation	An admin action that overrides the normal state machine: force-cancel, mark-paid-offline (record an external payment that didn't flow through BIL-01), write-off, override-overdue (release from OVERDUE), restore-from-cancelled. All recovery operations require role-gated permission and full audit.
Status event	A Kafka event emitted by STA-01 when a status changes: `InvoiceStatusChanged`, `InvoiceOverdue`, `InvoiceDisputed`, `InvoiceCancelled`, `InvoiceWrittenOff`, `PtpRegistered`, `PtpKept`, `PtpBroken`, `DisputeOpened`, `DisputeResolved`. Consumed downstream.

A — Target

1. What this process is about

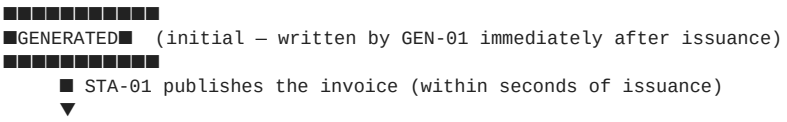
After GEN-01 / TAX-01 generates an invoice and writes it to the `invoice` table, that invoice enters a lifecycle. It needs to be paid, possibly disputed, possibly written off if uncollectable, possibly cancelled if generated in error. The state machine that governs this journey is STA-01.

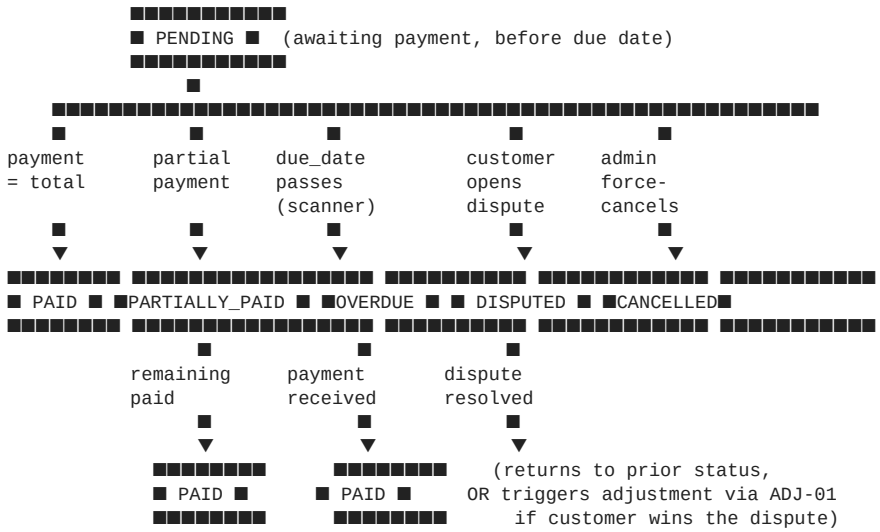
STA-01 has three responsibilities:

- **Automatic transitions**: respond to events (payment, due date passed) and move the invoice to the right status
- **Admin recovery operations**: support human-driven actions that don't fit the automatic flow (force-cancel, write-off, override)
- **PTPs and disputes**: manage the two structured exceptions where an invoice is in a special non-paid, non-overdue state for a defined reason

STA-01 does NOT touch payment amounts, balance computations, or wallet movements — those live in BIL-01 Charging Engine and BIL-05 Wallet and Topup. STA-01 listens for `PaymentApplied` events and moves status accordingly; the actual money flow is upstream.

The state machine



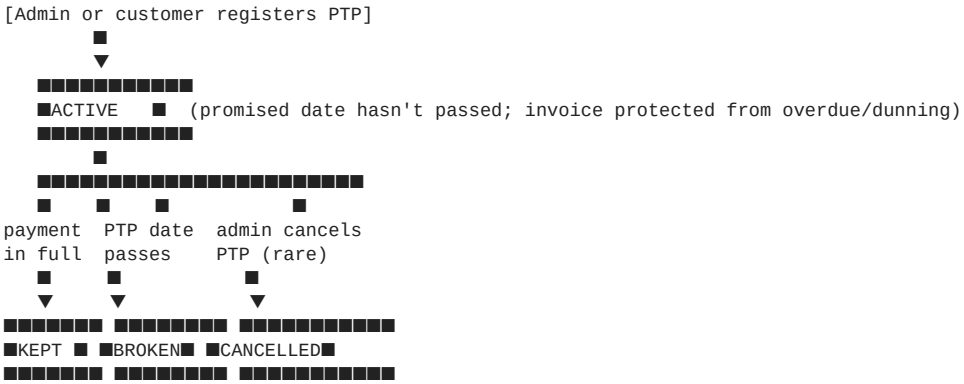


Three additional states reachable from multiple sources:

- **OVERDUE_PARTIAL**: an invoice was partially paid, due date passed, remaining amount is still owed. Treated like OVERDUE for collections purposes but distinguishable in reports.
- **WRITTEN_OFF**: the operator has decided the invoice is uncollectable. Admin action via recovery operation. Invoice is no longer pursued; remains in audit.
- **PENDING_PTP**: a sub-state of PENDING/OVERDUE indicating an active PTP exists. The overdue scanner skips invoices in this sub-state until the PTP date passes.

Promise-to-pay (first-class)

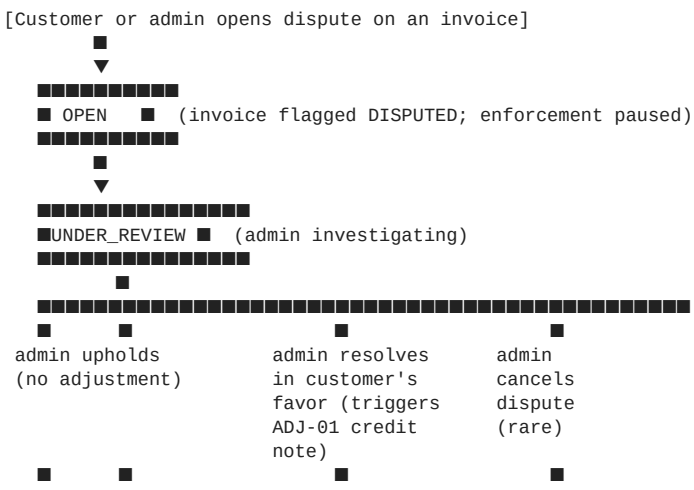
A PTP is a separate entity, not just a flag on the invoice. Lifecycle:

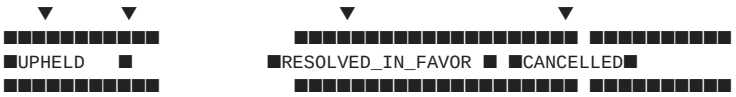


A PTP can be a single date+amount or multi-installment. While ACTIVE, the invoice is protected from OVERDUE transition and from dunning escalation (BIL-04 Collections honors `ptp_active = true` events).

Disputes (first-class)

A dispute is a separate entity. Lifecycle:





When a dispute is OPEN or UNDER_REVIEW, the underlying invoice is in DISPUTED status — protected from overdue scanner, protected from dunning. When the dispute resolves:

- **UPHELD**: invoice returns to its prior status (PENDING / OVERDUE / PARTIALLY_PAID)
- **RESOLVED_IN_FAVOR**: STA-01 triggers an ADJ-01 adjustment request (typically a credit note) with the dispute as the reason; invoice returns to prior status; the adjustment reduces what's owed
- **CANCELLED**: dispute withdrawn (customer dropped the complaint); invoice returns to prior status

Recovery operations (admin-driven)

Some situations require admin override of the automatic state machine. STA-01 exposes role-gated REST endpoints for these:

Operation	What it does	Typical use
Force-cancel	Move invoice to CANCELLED regardless of current status (except PAID — paid invoices need reversal via ADJ-01)	Invoice generated in error, before payment
Mark-paid-offline	Record an external payment that bypassed BIL-01 (cash payment to retail agent, bank transfer reconciled manually); moves invoice to PAID	External payment reconciliation
Write-off	Move invoice to WRITTEN_OFF; signals to BIL-04 Collections to stop pursuing	Customer bankrupt, deceased, uncollectable after exhausting collection efforts
Override-overdue	Move invoice from OVERDUE back to PENDING with a new due date	Operator extends payment deadline (negotiated grace period that doesn't fit the PTP model)
Restore-from-cancelled	Move CANCELLED invoice back to PENDING (with restored due date)	Cancellation was a mistake; needs reversal before any re-issue
Manual transition	Generic admin transition between allowed states with mandatory reason and approval	Exceptional cases not covered above

All recovery operations:

- Require a role with `RECOVERY_OPERATOR` permission (defined in FOUNDATION_AUTH.md)
- Require a mandatory reason (free text + reason code from operator catalog)
- Are logged in `invoice_history` AND `recovery_operation_log` (dedicated audit table)
- Emit a `RecoveryOperationApplied` event consumed by audit and analytics
- Some (write-off, mark-paid-offline above a threshold) require dual approval via Camunda — operator-configurable threshold per FOUNDATION_CAMUNDA.md

What STA-01 explicitly does not own

- **Invoice generation** — BIL-02-GEN-01 Invoice Generation
- **Tax invoice generation / signing** — BIL-02-TAX-01 Tax Invoice & Gateway
- **Adjustment proposal/approval workflow** — BIL-02-ADJ-01 Invoice Adjustments (STA-01 may TRIGGER an adjustment via ADJ-01 when a dispute resolves in favor of the customer, but doesn't own the workflow)
- **Read API and PDF retrieval** — BIL-02-READ-01 Invoice Read API
- **Payment recording itself** — BIL-01 Charging Engine (STA-01 consumes `PaymentApplied` and transitions status; BIL-01 owns the money flow)
- **Wallet operations** — BIL-05 Wallet and Topup
- **Dunning policy and escalation** — BIL-04 Collections (STA-01 emits `InvoiceOverdue`; BIL-04 decides the escalation path and emits dunning instructions to NOT-01)
- **Dunning communications** — DD_NOT-01 Notification Service

2. Business rules

Rules grouped by phase: `T` (transitions), `S` (scanners), `P` (PTP), `D` (disputes), `R` (recovery operations), `E` (event emission).

Rule group T — Transitions

ID	Rule	Triggered on
R-STA-01-T-1	At invoice issuance, GEN-01 / TAX-01 writes the invoice with `status = GENERATED`. STA-01 consumes the `InvoiceIssued` event and transitions to `PENDING` once downstream consistency is achieved (related rows committed, related documents linked). The transition typically happens within seconds. Until PENDING, the invoice is not yet active for payment or scanning.	InvoiceIssued
R-STA-01-T-2	When `PaymentApplied` arrives from BIL-01 Charging Engine, STA-01 reads the event's `applied_amount`. If `applied_amount` equals the invoice's `outstanding_amount` (before this payment): status → `PAID`. If less: status → `PARTIALLY_PAID`. The outstanding amount calculation is BIL-01's responsibility; STA-01 trusts the event payload.	PaymentApplied (from BIL-01)
R-STA-01-T-3	A subsequent `PaymentApplied` on a PARTIALLY_PAID invoice may fully clear it: if the new `outstanding_amount` becomes zero, status → PAID. Otherwise it stays PARTIALLY_PAID.	Subsequent PaymentApplied
R-STA-01-T-4	When `CreditNoteApplied` arrives from BIL-01 (issued by ADJ-01 → GEN-01, applied by BIL-01 to reduce outstanding), STA-01 re-evaluates status: if outstanding becomes zero, status → PAID; if remaining outstanding is reduced but non-zero, status stays (PENDING / OVERDUE / PARTIALLY_PAID depending on date).	CreditNoteApplied
R-STA-01-T-5	Allowed transitions are enforced via a state machine table (see "State machine" in technical design). Attempts to transition outside the allowed set (e.g., PAID → CANCELLED automatically) are rejected at the service layer and require a recovery operation.	All transitions
R-STA-01-T-6	Every transition writes a row to `invoice_history` (the audit shadow table owned by Core): `old_status`, `new_status`, `triggered_by` (event name or admin user ID), `reason_code`, `reason_text`, `transitioned_at`.	Every transition

Rule group S — Scanners

ID	Rule	Triggered on
R-STA-01-S-1	The **overdue scanner** runs every 15 minutes. It queries `invoice` for rows where `status IN (PENDING, PARTIALLY_PAID)` AND `due_date < now()` AND no active PTP AND no active dispute. For each match: PENDING → OVERDUE, PARTIALLY_PAID → OVERDUE_PARTIAL. Emits `InvoiceOverdue`.	Scheduled scanner
R-STA-01-S-2	The **PTP-expiry scanner** runs every 15 minutes. It queries `promise_to_pay` for rows where `status = ACTIVE` AND `promised_date < now()`. For each: checks the invoice's current outstanding; if zero → PTP transitions to KEPT (no action on invoice); if non-zero → PTP transitions to BROKEN, emits `PtpBroken`, and triggers a STA-01 re-evaluation that may transition the invoice to OVERDUE if past its due date.	Scheduled scanner
R-STA-01-S-3	The **dispute-aging scanner** runs daily. It queries `dispute` for rows where `status IN (OPEN, UNDER_REVIEW)` AND `opened_at < now() - dispute_aging_threshold` (operator-configurable, default 30 days). For matches: emits `DisputeAgingAlert` for admin visibility; does NOT auto-resolve.	Scheduled scanner
R-STA-01-S-4	Scanners are idempotent and resumable. Each row is processed in its own transaction; failures don't block subsequent rows. Scanners write a `scanner_run` audit row with row counts, start/end timestamps, and any errors.	Every scanner run

ID	Rule	Triggered on
R-STA-01-S-5	Scanners are operator-aware: each scanner iteration is scoped to one operator at a time, prevents cross-operator interference, and respects per-operator scanner enable/disable flags (some operators may not use overdue scanning).	Scanner scoping

Rule group P — Promise-to-pay

ID	Rule	Triggered on
R-STA-01-P-1	A PTP is registered via REST endpoint or admin console UI. Required fields: `invoice_id`, `promised_amount`, `promised_date`, `reason_code`, free-text justification. Optional: installments (list of `{amount, date}` for multi-step PTPs). The system validates: invoice exists and is in PENDING / PARTIALLY_PAID / OVERDUE; promised_date is in the future; promised_amount is ≤ outstanding amount; no other ACTIVE PTP exists for this invoice.	PTP registration
R-STA-01-P-2	On successful registration, PTP row is created with `status = ACTIVE`. The invoice's effective sub-state is now PENDING_PTP (or OVERDUE_PTP if already overdue) — surfaced as a `ptp_active = true` flag in event emissions and in the read API. Emits `PtpRegistered`.	Registration commit
R-STA-01-P-3	While a PTP is ACTIVE: the overdue scanner skips this invoice; BIL-04 Collections honors `PtpRegistered` and pauses dunning escalation; STA-01 does not transition the invoice based on due_date passing.	PTP active period
R-STA-01-P-4	When a payment is applied that brings outstanding to zero, STA-01 emits `PaymentApplied`-driven transition AND emits `PtpKept` on the related PTP. PTP status → KEPT.	Payment during PTP
R-STA-01-P-5	When the PTP-expiry scanner determines the PTP date has passed without sufficient payment: PTP status → BROKEN; emit `PtpBroken`; the invoice is now eligible for overdue transition on the next overdue-scanner run (and BIL-04 Collections will resume dunning escalation).	PTP expiry without payment
R-STA-01-P-6	An admin can manually cancel an ACTIVE PTP (e.g., customer requests cancellation). PTP status → CANCELLED; emit `PtpCancelled`; invoice's protection lifts; next overdue scanner pass reassesses. Requires `PTP_MANAGER` role.	Manual PTP cancellation
R-STA-01-P-7	Multi-installment PTPs: a single PTP row with `installments` JSONB array; each installment has its own `due_date` and `amount`. The PTP scanner checks each installment's `due_date`; if any installment is missed without sufficient payment toward it, the PTP transitions to BROKEN immediately (whole PTP, not just the missed installment — operator policy default; configurable to "tolerate one missed installment" per operator).	Installment tracking

Rule group D — Disputes

ID	Rule	Triggered on
R-STA-01-D-1	A dispute is opened via REST endpoint or admin console UI. Required fields: `invoice_id`, `dispute_reason_code`, free-text justification, optional `disputed_amount` (if partial), optional `disputed_line_item_ids` (if specific lines). Opener can be customer (via self-service portal) or admin (on customer's behalf via support call). System validates: invoice exists, is in PENDING / PARTIALLY_PAID / OVERDUE / OVERDUE_PARTIAL; no other OPEN/UNDER_REVIEW dispute exists for this invoice; if disputed_amount is given, it's ≤ outstanding.	Dispute opening

ID	Rule	Triggered on
R-STA-01-D-2	On open, dispute row created with `status = OPEN`. Invoice status → DISPUTED (state machine allows this from PENDING / PARTIALLY_PAID / OVERDUE / OVERDUE_PARTIAL). The prior status is captured in `invoice.dispute_prior_status` so it can be restored. Emits `DisputeOpened` and `InvoiceDisputed`.	Open commit
R-STA-01-D-3	While the dispute is OPEN or UNDER_REVIEW: BIL-04 Collections pauses dunning escalation; overdue scanner skips this invoice; reports flag it as disputed for operational visibility.	Dispute active period
R-STA-01-D-4	An admin (role `DISPUTE_RESOLVER`) moves the dispute from OPEN → UNDER_REVIEW when they start investigating. Captures the admin user ID and timestamp. Emits `DisputeUnderReview`.	Admin starts review
R-STA-01-D-5	**Resolution — UPHELD**: admin concludes the invoice is correct. Dispute → UPHELD. Invoice returns to `dispute_prior_status` (PENDING / OVERDUE / etc.) with a fresh due_date if the operator policy extends due dates after resolution (configurable). Emits `DisputeResolvedUpheld`. The invoice is now eligible for the normal flow again (overdue scanner, dunning).	UPHELD resolution
R-STA-01-D-6	**Resolution — RESOLVED_IN_FAVOR**: admin concludes the customer is right. Dispute → RESOLVED_IN_FAVOR. STA-01 triggers an ADJ-01 adjustment request (typically a credit note) with the dispute_id as reference, the disputed_amount as the proposed credit, and the dispute reason_code mapped to an ADJ-01 reason_code. The ADJ-01 workflow then proceeds (proposal → approval → applied). Invoice returns to `dispute_prior_status`. Emits `DisputeResolvedInFavor`.	RESOLVED_IN_FAVOR resolution
R-STA-01-D-7	**Resolution — CANCELLED**: dispute withdrawn (customer dropped the complaint, or admin determines it was opened in error). Dispute → CANCELLED. Invoice returns to `dispute_prior_status`. Emits `DisputeCancelled`.	CANCELLED resolution
R-STA-01-D-8	After resolution, no further actions allowed on the dispute. If the customer disputes the same invoice again, a new dispute row is opened (with reference to the previous dispute_id for context).	Post-resolution

Rule group R — Recovery operations

ID	Rule	Triggered on
R-STA-01-R-1	**Force-cancel** (`POST /api/admin/invoices/{id}/force-cancel`): allowed only for invoices in PENDING / PARTIALLY_PAID / OVERDUE / OVERDUE_PARTIAL / DISPUTED (not PAID, not already CANCELLED, not WRITTEN_OFF). Moves invoice → CANCELLED. Mandatory reason_code + free-text justification. Captures admin user ID. Emits `InvoiceCancelled` with reason.	Admin force-cancel
R-STA-01-R-2	**Mark-paid-offline** (`POST /api/admin/invoices/{id}/mark-paid-offline`): allowed for invoices in PENDING / PARTIALLY_PAID / OVERDUE / OVERDUE_PARTIAL. Records an external payment (cash to agent, bank transfer reconciled manually) by emitting a `PaymentReceived` to BIL-01 with `source = OFFLINE_RECORDED`. BIL-01 records and confirms via `PaymentApplied`; STA-01 transitions the invoice normally. Above a per-operator threshold (default 50,000 KES equivalent), requires dual approval via Camunda.	Admin mark-paid-offline

ID	Rule	Triggered on
R-STA-01-R-3	**Write-off** (`POST /api/admin/invoices/{id}/write-off`): allowed for invoices in OVERDUE / OVERDUE_PARTIAL. Moves invoice → WRITTEN_OFF. Mandatory reason_code + free-text. **Always requires dual approval** via Camunda (write-offs are accounting events that need finance sign-off). Emits `InvoiceWrittenOff`; BIL-04 Collections stops pursuing; BIL-01 keeps the outstanding amount recorded for audit but flagged as written-off.	Admin write-off
R-STA-01-R-4	**Override-overdue** (`POST /api/admin/invoices/{id}/override-overdue`): allowed for invoices in OVERDUE / OVERDUE_PARTIAL. Moves back to PENDING / PARTIALLY_PAID with a new `due_date` provided by the admin. Use case: customer negotiated an extension that doesn't fit the PTP model (e.g., operator policy extends due date for a whole region after a service outage). Mandatory reason_code. Emits `InvoiceOverdueOverridden`.	Admin override-overdue
R-STA-01-R-5	**Restore-from-cancelled** (`POST /api/admin/invoices/{id}/restore`): allowed only for invoices in CANCELLED. Moves back to PENDING with a restored or new due_date. Use case: cancellation was an error. Mandatory reason_code. Always requires dual approval. Emits `InvoiceRestored`.	Admin restore
R-STA-01-R-6	**Manual transition** (`POST /api/admin/invoices/{id}/transition`): generic admin transition. Allowed transitions are whitelisted per operator config (e.g., some operators may allow PARTIALLY_PAID → PENDING in specific cases). Mandatory reason_code + dual approval. Emits `InvoiceManualTransition`.	Admin generic transition
R-STA-01-R-7	All recovery operations write to `invoice_history` AND `recovery_operation_log` (dedicated table). Operations requiring dual approval go through a Camunda BPMN per FOUNDATION_CAMUNDA.md, similar to the ADJ-01 approval pattern.	Audit

Rule group E — Event emission

ID	Rule	Triggered on
R-STA-01-E-1	Every status transition emits an `InvoiceStatusChanged` event with: `invoice_id`, `customer_id`, `operator_code`, `old_status`, `new_status`, `triggered_by`, `reason_code`, `outstanding_amount_after`, `transitioned_at`. Consumed by READ-01 (cache invalidation), BIL-04 Collections (escalation decisions), analytics.	Every transition
R-STA-01-E-2	Specific transitions also emit named events for clarity: `InvoiceOverdue`, `InvoicePaid`, `InvoiceCancelled`, `InvoiceWrittenOff`, `InvoiceDisputed`, `InvoicePartiallyPaid`. These are convenience events on a separate Kafka topic for consumers that don't want to filter generic status-change events.	Specific transitions
R-STA-01-E-3	PTP events: `PtpRegistered`, `PtpKept`, `PtpBroken`, `PtpCancelled`. Payload includes the PTP entity and the related invoice_id. Consumed by BIL-04 Collections (dunning pause/resume), DD_NOT-01 (customer confirmation messages), analytics.	PTP lifecycle
R-STA-01-E-4	Dispute events: `DisputeOpened`, `DisputeUnderReview`, `DisputeResolvedUpheld`, `DisputeResolvedInFavor`, `DisputeCancelled`, `DisputeAgingAlert`. Payload includes the dispute entity, the invoice_id, and resolution details.	Dispute lifecycle

ID	Rule	Triggered on
R-STA-01-E-5	Recovery events: `InvoiceManualTransition`, `InvoiceOverdueOverridden`, `InvoiceRestored`, `RecoveryOperationApplied`. Always include the admin user ID, reason_code, and timestamps.	Recovery operations

3. Module owner and impacted modules

Role	Module	What it does
Owner	New `billing-invoicing-status` package within the `billing-invoicing` module on cluster C	Owns the state machine service, the scanners, the PTP and dispute aggregates, the recovery REST endpoints, the Camunda integration for dual-approval recovery operations.
Camunda workflow engine	Existing Camunda deployment (per FOUNDATION_CAMUNDA.md)	Runs dual-approval workflows for write-off, mark-paid-offline above threshold, restore-from-cancelled, manual transition.
Upstream events consumed	BIL-01 Charging Engine (`PaymentApplied`, `CreditNoteApplied`), BIL-02-GEN-01 / TAX-01 (`InvoiceIssued`, `TaxInvoiceSigned`)	Drive automatic transitions
Downstream events emitted	`InvoiceStatusChanged`, `InvoiceOverdue`, `InvoiceDisputed`, `InvoiceCancelled`, `InvoiceWrittenOff`, `PtpRegistered`, `PtpKept`, `PtpBroken`, `DisputeOpened`, `DisputeResolved*`, `RecoveryOperationApplied`, `DisputeAgingAlert`	Consumed by READ-01, BIL-04 Collections, DD_NOT-01, analytics
Frontend	Admin console — recovery operations dashboard, PTP management UI, dispute resolution queue, scanner monitoring	Built on Core's admin shell
Customer self-service	Customer portal — open dispute, view dispute status	Frontend consumes STA-01's customer-facing REST endpoints
Engines	Camunda for dual-approval recovery workflows. No Drools — state transitions are simple enough to keep in code.	—

4. Foundation references

DB → `FOUNDATION_DB.md` · Kafka → `FOUNDATION_KAFKA.md` · Camunda → `FOUNDATION_CAMUNDA.md` · Auth → `FOUNDATION_AUTH.md` (introduces `PTP_MANAGER`, `DISPUTE_RESOLVER`, `RECOVERY_OPERATOR` roles). Cache → `FOUNDATION_CACHE.md` (STA-01 does not cache directly; status reads are served via READ-01 which caches).

Technical design

Module placement

Co-located in `billing-invoicing` module on cluster C. New sub-package `status`. Adds 4 new dedicated tables (`promise_to_pay`, `dispute`, `recovery_operation_log`, `scanner_run`).

Code structure

```
modules/billing-invoicing/
  src/main/java/com/sophix/billing/invoicing/status/
    service/
      InvoiceStatusService.java         ← state machine — transition orchestration
      OverdueScanner.java               ← scheduled 15-minute scanner
      PtpExpiryScanner.java             ← scheduled 15-minute scanner
      DisputeAgingScanner.java          ← scheduled daily scanner
      PromiseToPayService.java          ← PTP lifecycle
      DisputeService.java               ← dispute lifecycle
      RecoveryOperationService.java     ← admin recovery actions
    camunda/
      RecoveryApprovalDelegate.java     ← Camunda service tasks for dual approval
    kafka/
      PaymentEventConsumer.java         ← consumes PaymentApplied / CreditNoteApplied
      InvoiceIssuedEventConsumer.java   ← consumes InvoiceIssued (transition GENERATED → PENDING)
    rest/
      PtpResource.java                  ← customer + admin PTP endpoints
```

```
    DisputeResource.java             ← customer + admin dispute endpoints
    RecoveryResource.java            ← admin recovery endpoints
    StatusAdminResource.java         ← scanner monitoring, reporting
  domain/
    InvoiceStatusTransition.java     ← state machine rules (Java enum + transition table)
    PromiseToPay.java
    Dispute.java
    RecoveryOperation.java
    ScannerRun.java
  repository/
    PromiseToPayRepository.java
    DisputeRepository.java
    RecoveryOperationLogRepository.java
    ScannerRunRepository.java
src/main/resources/
  config/liquibase/changelog/billing-invoicing/
    080_promise_to_pay.xml
    081_dispute.xml
    082_recovery_operation_log.xml
    083_scanner_run.xml
  bpmn/
    recovery_approval_writeoff.bpmn
    recovery_approval_offline_payment.bpmn
    recovery_approval_restore.bpmn
    recovery_approval_manual_transition.bpmn
```

State machine

The set of allowed transitions, encoded as a table (mirrored in code as a `Map<Pair<Status, Status>, TransitionRule>`):

From	To	Trigger	Allowed?
GENERATED	PENDING	`InvoiceIssued` consumed	✓ (automatic)
PENDING	PAID	`PaymentApplied` with full amount	✓ (automatic)
PENDING	PARTIALLY_PAID	`PaymentApplied` with partial amount	✓ (automatic)
PENDING	OVERDUE	overdue scanner	✓ (automatic; skipped if active PTP or dispute)
PENDING	DISPUTED	dispute opened	✓ (automatic on dispute open)
PENDING	CANCELLED	force-cancel	✓ (admin recovery)
PARTIALLY_PAID	PAID	subsequent `PaymentApplied` or `CreditNoteApplied` clearing outstanding	✓ (automatic)
PARTIALLY_PAID	OVERDUE_PARTIAL	overdue scanner	✓ (automatic; skipped if active PTP or dispute)
PARTIALLY_PAID	DISPUTED	dispute opened	✓ (automatic on dispute open)
PARTIALLY_PAID	CANCELLED	force-cancel	✓ (admin recovery)
OVERDUE	PAID	`PaymentApplied` with full amount	✓ (automatic)
OVERDUE	PARTIALLY_PAID	`PaymentApplied` with partial amount	✓ (automatic)
OVERDUE	OVERDUE_PARTIAL	`PaymentApplied` with partial amount	✓ (automatic, alt path)
OVERDUE	DISPUTED	dispute opened	✓ (automatic on dispute open)
OVERDUE	PENDING	override-overdue with new due_date	✓ (admin recovery)
OVERDUE	CANCELLED	force-cancel	✓ (admin recovery)
OVERDUE	WRITTEN_OFF	write-off (dual approval)	✓ (admin recovery)
OVERDUE_PARTIAL	PAID	`PaymentApplied` or `CreditNoteApplied` clearing remaining	✓ (automatic)
OVERDUE_PARTIAL	OVERDUE	(if next partial payment doesn't fully clear)	— same status semantics, kept as OVERDUE_PARTIAL until paid
OVERDUE_PARTIAL	DISPUTED	dispute opened	✓ (automatic on dispute open)
OVERDUE_PARTIAL	PARTIALLY_PAID	override-overdue with new due_date	✓ (admin recovery)
OVERDUE_PARTIAL	CANCELLED	force-cancel	✓ (admin recovery)
OVERDUE_PARTIAL	WRITTEN_OFF	write-off (dual approval)	✓ (admin recovery)
DISPUTED	<prior status>	dispute resolved (UPHELD or CANCELLED)	✓ (automatic — uses `dispute_prior_status`)

From	To	Trigger	Allowed?
DISPUTED	<prior status, with ADJ-01 in motion>	dispute RESOLVED_IN_FAVOR	✓ (automatic — returns to prior; ADJ-01 credit reduces outstanding separately)
CANCELLED	PENDING	restore-from-cancelled (dual approval)	✓ (admin recovery)
PAID	*	(any transition out)	✗ — paid invoices need ADJ-01 credit note for reversal, not status change
WRITTEN_OFF	*	(any transition out)	✗ — written-off is terminal except for accounting reversal via ADJ-01

Disallowed transitions return `409 Conflict` with `error_code = STATUS_TRANSITION_NOT_ALLOWED`.

Scanner flows

Overdue scanner (every 15 minutes)

```
Step 1 — Query candidates
  SELECT id, operator_code, customer_id, status, due_date, outstanding_amount
  FROM invoice
  WHERE status IN ('PENDING', 'PARTIALLY_PAID')
    AND due_date < NOW()
    AND id NOT IN (SELECT invoice_id FROM promise_to_pay WHERE status = 'ACTIVE')
    AND id NOT IN (SELECT invoice_id FROM dispute WHERE status IN ('OPEN', 'UNDER_REVIEW'))
    AND operator_code IN (operators with overdue_scanner_enabled = true)
  ORDER BY due_date ASC
  LIMIT 1000;

Step 2 — Process each row (independent transactions)
  For each candidate:
    BEGIN TRANSACTION
      SELECT status FROM invoice WHERE id = ? FOR UPDATE
      IF status IN ('PENDING', 'PARTIALLY_PAID') AND due_date < NOW():
        new_status = (status = 'PENDING') ? 'OVERDUE' : 'OVERDUE_PARTIAL'
        UPDATE invoice SET status = new_status, updated_at = NOW()
        INSERT INTO invoice_history (invoice_id, old_status, new_status, triggered_by, transitioned_at)
        emit InvoiceStatusChanged + InvoiceOverdue events
    COMMIT

Step 3 — Loop until no more candidates
  If 1000 rows returned, query again; else stop

Step 4 — Write scanner_run row
  INSERT INTO scanner_run (scanner_name, operator_code, started_at, ended_at, rows_processed, errors_count)
```

Scanner failures on individual rows do not block the run. If a transaction fails (e.g., row was updated by another process), it's skipped and counted as an error; the run continues.

PTP-expiry scanner (every 15 minutes)

```
Step 1 — Query candidates
  SELECT id, invoice_id, promised_date, promised_amount
  FROM promise_to_pay
  WHERE status = 'ACTIVE'
    AND promised_date < NOW();

Step 2 — Process each
  For each PTP:
    Look up invoice's current outstanding_amount
    If outstanding_amount == 0:
      PTP status → KEPT
      Emit PtpKept
    Else:
      PTP status → BROKEN
      Emit PtpBroken
      (Invoice will be re-evaluated by next overdue scanner pass)

Step 3 — Multi-installment check
  For PTPs with installments JSONB:
    For each installment due in the past:
      If insufficient payment toward it: PTP → BROKEN immediately
      (Per operator config: tolerate one missed installment or strict)

Step 4 — Write scanner_run row
```

Dispute-aging scanner (daily)

Queries disputes in OPEN/UNDER_REVIEW older than threshold; emits `DisputeAgingAlert` for admin visibility; does not modify status.

Data model

`promise_to_pay`

Column	Type	Notes
`id`	TEXT PK	ULID
`operator_code`	TEXT NOT NULL	
`invoice_id`	TEXT NOT NULL	FK to `invoice`
`customer_id`	TEXT NOT NULL	Denormalized for lookup
`promised_amount`	NUMERIC(18,4) NOT NULL	
`promised_date`	DATE NOT NULL	Single-date PTP
`installments`	JSONB NULL	Multi-installment PTPs: `[{amount, date}]`
`currency`	TEXT NOT NULL	
`reason_code`	TEXT NOT NULL	From operator catalog
`justification`	TEXT NOT NULL	Free text
`status`	TEXT NOT NULL	`ACTIVE` / `KEPT` / `BROKEN` / `CANCELLED`
`registered_by`	TEXT NOT NULL	Admin user ID or customer_id
`registered_at`	TIMESTAMPTZ NOT NULL	
`resolved_at`	TIMESTAMPTZ NULL	When transitioned out of ACTIVE
`resolution_event`	TEXT NULL	`PAYMENT_RECEIVED` / `DATE_PASSED_UNPAID` / `MANUAL_CANCEL`
`metadata`	JSONB NULL	Operator-specific extensions

Indexes: `(invoice_id) WHERE status = 'ACTIVE'` UNIQUE (prevents two active PTPs per invoice), `(operator_code, status, promised_date)`, `(customer_id, registered_at DESC)`. P10Y retention.

Sample data:

id	invoice_id	promised_amount	promised_date	status	resolution_event
`ptp_01HZ...A`	`inv_01HZ...`	6402.50	2026-05-25	`ACTIVE`	—
`ptp_01HZ...B`	`inv_01HZ...`	3000.00	2026-05-10	`KEPT`	`PAYMENT_RECEIVED`
`ptp_01HZ...C`	`inv_01HZ...`	5000.00	2026-05-08	`BROKEN`	`DATE_PASSED_UNPAID`

`dispute`

Column	Type	Notes
`id`	TEXT PK	ULID
`operator_code`	TEXT NOT NULL	
`invoice_id`	TEXT NOT NULL	FK to `invoice`
`customer_id`	TEXT NOT NULL	Denormalized
`disputed_amount`	NUMERIC(18,4) NULL	If partial; NULL means whole invoice
`disputed_line_item_ids`	JSONB NULL	If specific lines
`reason_code`	TEXT NOT NULL	From operator catalog
`justification`	TEXT NOT NULL	Free text
`status`	TEXT NOT NULL	`OPEN` / `UNDER_REVIEW` / `UPHELD` / `RESOLVED_IN_FAVOR` / `CANCELLED`
`opened_by`	TEXT NOT NULL	Admin user ID or customer_id
`opened_at`	TIMESTAMPTZ NOT NULL	
`review_started_by`	TEXT NULL	When status → UNDER_REVIEW
`review_started_at`	TIMESTAMPTZ NULL	
`resolved_by`	TEXT NULL	When status moves to terminal
`resolved_at`	TIMESTAMPTZ NULL	

Column	Type	Notes
resolution_notes	TEXT NULL	Admin's explanation
triggered_adjustment_request_id	TEXT NULL	If RESOLVED_IN_FAVOR, the ADJ-01 request created
previous_dispute_id	TEXT NULL	If this is a re-dispute (FK to previous resolved dispute on same invoice)
metadata	JSONB NULL	Operator-specific extensions

Indexes: (invoice_id) WHERE status IN ('OPEN', 'UNDER_REVIEW') UNIQUE, (operator_code, status, opened_at DESC), (customer_id, opened_at DESC). P10Y retention.

recovery_operation_log

One row per recovery operation. Append-only.

Column	Type	Notes
id	TEXT PK	ULID
invoice_id	TEXT NOT NULL	FK
operation_type	TEXT NOT NULL	FORCE_CANCEL / MARK_PAID_OFFLINE / WRITE_OFF / OVERRIDE_OVERDUE / RESTORE / MANUAL_TRANSITION
requested_by	TEXT NOT NULL	Admin user ID
reason_code	TEXT NOT NULL	
justification	TEXT NOT NULL	
prior_status	TEXT NOT NULL	Invoice status before operation
new_status	TEXT NOT NULL	Invoice status after operation
requires_approval	BOOLEAN NOT NULL	Whether dual approval was needed
camunda_process_instance_id	TEXT NULL	If approval workflow ran
approvers	JSONB NULL	Array of {user_id, action, at} for the approval chain
requested_at	TIMESTAMPTZ NOT NULL	
applied_at	TIMESTAMPTZ NULL	When operation was actually executed (after approval if needed)
metadata	JSONB NULL	E.g., new due_date for override-overdue, mark-paid-offline payment_reference

Indexes: (invoice_id, requested_at DESC), (operation_type, requested_at DESC), (requested_by, requested_at DESC). P10Y retention.

scanner_run

One row per scanner execution. Used for monitoring and debugging.

Column	Type	Notes
id	TEXT PK	ULID
scanner_name	TEXT NOT NULL	OVERDUE_SCANNER / PTP_EXPIRY_SCANNER / DISPUTE_AGING_SCANNER
operator_code	TEXT NULL	Per-operator scanner runs
started_at	TIMESTAMPTZ NOT NULL	
ended_at	TIMESTAMPTZ NULL	
rows_processed	INTEGER NOT NULL DEFAULT 0	
rows_transitioned	INTEGER NOT NULL DEFAULT 0	Subset that actually changed status
errors_count	INTEGER NOT NULL DEFAULT 0	
error_summary	JSONB NULL	If errors occurred

Indexes: (scanner_name, started_at DESC), (operator_code, started_at DESC). P90D retention (scanner audit is high-volume; finer-grained logging is in the application logs).

API surface

Mechanism	Used for	Examples
REST (customer-facing)	Customer opens dispute, views dispute status, views own PTPs	`POST /api/disputes`, `GET /api/disputes/{id}`, `GET /api/ptps?customer=me`
REST (admin-facing)	Recovery operations, PTP management, dispute resolution, scanner monitoring	`POST /api/admin/invoices/{id}/force-cancel`, `POST /api/admin/disputes/{id}/resolve`, `GET /api/admin/scanners/runs`
Kafka consumption	Driving automatic transitions	`PaymentApplied`, `CreditNoteApplied`, `InvoiceIssued`, `TaxInvoiceSigned`
Kafka emission	Status changes, PTP/dispute lifecycle, recovery operations	`InvoiceStatusChanged`, `InvoiceOverdue`, `PtpRegistered`, `DisputeOpened`, `RecoveryOperationApplied`, etc.
Camunda BPMN	Dual approval for write-off / mark-paid-offline / restore / manual transition	Process instances per recovery operation
In-process Java call	Triggering ADJ-01 when dispute RESOLVED_IN_FAVOR	`AdjustmentRequestService.proposeFromDisputeResolution(...)`

REST endpoints

```
# Customer-facing
POST   /api/disputes                          ← open dispute
GET    /api/disputes/{id}                     ← view dispute (own only)
GET    /api/disputes?invoice_id={id}          ← list disputes for an invoice (own only)
GET    /api/ptps?invoice_id={id}              ← list PTPs for an invoice (own only)
GET    /api/ptps/{id}                         ← view PTP detail (own only)

# Admin-facing — PTP
POST   /api/admin/ptps                        ← register PTP on customer's behalf
POST   /api/admin/ptps/{id}/cancel            ← manually cancel active PTP

# Admin-facing — Dispute
POST   /api/admin/disputes/{id}/start-review  ← move OPEN → UNDER_REVIEW
POST   /api/admin/disputes/{id}/resolve       ← resolve (UPHELD / RESOLVED_IN_FAVOR / CANCELLED)

# Admin-facing — Recovery
POST   /api/admin/invoices/{id}/force-cancel
POST   /api/admin/invoices/{id}/mark-paid-offline
POST   /api/admin/invoices/{id}/write-off
POST   /api/admin/invoices/{id}/override-overdue
POST   /api/admin/invoices/{id}/restore
POST   /api/admin/invoices/{id}/transition

# Admin-facing — Monitoring
GET    /api/admin/scanners/runs               ← scanner execution history
GET    /api/admin/scanners/{name}/status      ← last run + next scheduled run
POST   /api/admin/scanners/{name}/trigger     ← manually trigger a scanner run (ops use)
```

Request / response samples

Open dispute (customer)

Request:

```
POST /api/disputes HTTP/1.1
Host: api.sophix.example
Authorization: Bearer <customer-JWT>
Content-Type: application/json

{
  "invoice_id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "reason_code": "CHARGE_INCORRECT",
  "justification": "I was charged for premium TV package but I only subscribed to basic.",
  "disputed_amount": 2900.30,
  "disputed_line_item_ids": ["il_01HZ..."]
}
```

Response (201 Created):

```
{
  "id": "dsp_01HZ...",
  "invoice_id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "status": "OPEN",
  "disputed_amount": 2900.30,
  "reason_code": "CHARGE_INCORRECT",
  "opened_at": "2026-05-18T11:23:45Z",
  "message": "Dispute opened. Your invoice is now in DISPUTED status. We will review and respond within 14 business days."
}
```

The invoice's status flips to DISPUTED at the same time; READ-01 cache is invalidated; BIL-04 Collections pauses dunning escalation.

Resolve dispute in customer's favor (admin)

Request:

```
POST /api/admin/disputes/dsp_01HZ.../resolve HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: application/json

{
  "resolution": "RESOLVED_IN_FAVOR",
  "resolution_notes": "Verified customer subscribed to basic; premium charge was system error during package change.",
  "credit_amount": 2900.30,
  "credit_reason_code": "BILLING_ERROR"
}
```

Response (200 OK):

```
{
  "id": "dsp_01HZ...",
  "status": "RESOLVED_IN_FAVOR",
  "resolved_at": "2026-05-19T14:12:30Z",
  "resolved_by": "admin_user_42",
  "triggered_adjustment_request_id": "adj_01HZ...",
  "invoice_status_after": "OVERDUE",
  "message": "Dispute resolved in customer's favor. Adjustment request adj_01HZ... created for review and approval via ADJ
}
```

Register PTP (admin)

Request:

```
POST /api/admin/ptps HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: application/json

{
  "invoice_id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "promised_amount": 6402.50,
  "promised_date": "2026-05-25",
  "reason_code": "CUSTOMER_REQUESTED_EXTENSION",
  "justification": "Customer called CS, requested 10 days to pay due to salary delay."
}
```

Response (201 Created):

```
{
  "id": "ptp_01HZ...",
  "invoice_id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "promised_amount": 6402.50,
  "promised_date": "2026-05-25",
  "status": "ACTIVE",
  "registered_at": "2026-05-18T11:45:00Z",
  "registered_by": "admin_user_17",
  "message": "PTP registered. Invoice protected from overdue transition and dunning escalation until 2026-05-25."
}
```

Write-off (admin — dual approval)

Request:

```
POST /api/admin/invoices/inv_01HZ.../write-off HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: application/json

{
  "reason_code": "UNCOLLECTABLE_DECEASED",
  "justification": "Customer deceased, no estate to pursue per court records dated 2026-04-10."
}
```

Response (202 Accepted — workflow started):

```
{
  "recovery_operation_id": "rec_01HZ...",
  "operation_type": "WRITE_OFF",
  "status": "PENDING_APPROVAL",
  "camunda_process_instance_id": "proc_01HZ...",
  "approvers_pending": ["finance_manager"],
  "requested_at": "2026-05-18T13:00:00Z",
  "message": "Write-off requested. Awaiting finance manager approval before applying."
}
```

After approval, a follow-up event marks the invoice WRITTEN_OFF.

Cross-DD coordination

Touched	What STA-01 owns	What the other DD owns
BIL-02 Invoicing (Core)	Reads/writes `invoice.status`, writes `invoice_history`. Reuses Core's customer_snapshot data when emitting events.	Owns the `invoice` table schema, the status enum, the history table.
BIL-02-GEN-01 Invoice Generation	Consumes `InvoiceIssued` to drive GENERATED → PENDING.	Owns generation.
BIL-02-TAX-01 Tax Invoice & Gateway	Consumes `TaxInvoiceSigned` and treats it similarly (GENERATED → PENDING for tax invoices).	Owns tax invoice lifecycle.
BIL-02-ADJ-01 Invoice Adjustments	Triggers ADJ-01 adjustment request when a dispute resolves in customer's favor.	Owns the adjustment workflow itself.
BIL-02-READ-01 Invoice Read API	Emits status events that READ-01 consumes for cache invalidation.	Owns the read API.
BIL-01 Charging Engine	Consumes `PaymentApplied` (drives PENDING → PAID/PARTIALLY_PAID) and `CreditNoteApplied` (drives PARTIALLY_PAID → PAID if outstanding clears). Emits `PaymentReceived` for offline payment recording.	Owns money flow, balance updates.
BIL-04 Collections	Emits `InvoiceOverdue`, `PtpRegistered`, `PtpBroken`, `DisputeOpened`, `DisputeResolved*`. BIL-04 consumes and decides dunning escalation.	Owns dunning policy, escalation, service-level restrictions.
DD_NOT-01 Notification Service	Emits PTP / dispute events that NOT-01 may use to send customer confirmations.	Owns the actual notification channels.
Camunda Workflow Engine	Runs dual-approval BPMN for write-off, mark-paid-offline (above threshold), restore, manual transition.	Camunda is the runtime.
CRM / Customer Service	Customer-facing dispute/PTP endpoints are surfaced to support agents and to the customer portal.	Owns customer master, channel integrations.
Analytics	Consumes all status events for reporting (overdue rates, dispute volumes by reason, PTP keep rate, write-off rates).	Analytics platform.

Dependencies

Dependency	Owner
BIL-02 Invoicing (Core)	billing team (same wave)
BIL-02-GEN-01 Invoice Generation	billing team (same wave)
BIL-02-TAX-01 Tax Invoice & Gateway	billing team (same wave)
BIL-02-ADJ-01 Invoice Adjustments	billing team (same wave)
BIL-02-READ-01 Invoice Read API	billing team (same wave)
BIL-01 Charging Engine	billing team
BIL-04 Collections	billing team
DD_NOT-01 Notification Service	notification team
Camunda deployment	infrastructure team
FOUNDATION_DB.md, FOUNDATION_KAFKA.md, FOUNDATION_CAMUNDA.md, FOUNDATION_AUTH.md	infrastructure team
Per-operator reason code catalogs (dispute reasons, PTP reasons, recovery reasons)	each operator's billing/CS team
Per-operator dual-approval BPMN files	each operator's finance team
Per-operator scanner enable/disable flags + dispute aging threshold	each operator's ops team

Operator-configurable capabilities (specific to STA-01)

Capability	Where configured	Notes
Overdue scanner enabled	Per-operator config	Some operators may run their own external overdue logic and want STA-01's scanner off
Overdue scanner interval	Per-operator config	Default 15 minutes; some operators may want hourly
Dispute aging threshold	Per-operator config	Default 30 days; operators with tighter SLAs may set 14
PTP multi-installment policy	Per-operator config	Strict (one miss = BROKEN) or tolerant (one miss tolerated)
Mark-paid-offline threshold for dual approval	Per-operator config	Default 50,000 KES-equivalent; above this requires dual approval
Recovery operation BPMN refs	Per-operator deployment	Each operator deploys its own approval workflows for write-off, mark-paid-offline, restore, manual transition
Dispute reason code catalog	`dispute_reason_code` table per operator	Operator defines their dispute reasons with locale-aware display names
PTP reason code catalog	`ptp_reason_code` table per operator	Same pattern as dispute reasons
Recovery reason code catalog	`recovery_reason_code` table per operator	Reasons for force-cancel, write-off, etc.
Due-date extension after dispute resolution	Per-operator config	When a dispute resolves UPHELD, should the original due date stand or be extended by N days? Default 0 (no extension).

B — Current TZ

Workshop materials do not describe a unified status management or recovery service in TZ Sophix. There is no first-class state machine, no formal PTP entity, no formal dispute entity, and no documented scanner inventory to compare against.

C — Gap & bridge

STA-01 is entirely new functionality as a structured service. The state machine, first-class PTP and dispute entities, scanners, and role-gated recovery operations described in section A are introduced fresh by V3.